

Queen Mary University of London

School of Engineering and Materials Science

ECS414U — Object Oriented Programming

Horse Race Simulation

Balazs M Lovasz – Student ID [240252063]

[28/04/2025]

Table of Contents

Table of Contents

Table of Contents	1
Introduction	1
Task 1: Horse Class Implementation	1
Summary:.....	2
Encapsulation:	3
Testing and Results:	3
Task 2: Race Class Improvements	5
Review of the Original Race Class:	5
Issues Identified in the Original Race Class:	6
Improvements made:.....	6
Updated code:	6
Testing and Outputs:	7
Conclusion	9
Appendix	9

Introduction

This report outlines the development of a horse racing simulation program, where the development of the code was divided into two key tasks. The first task focused on the implementation of a Horse class with encapsulated data and clearly defined behaviours. The second involved improving a pre-written Race class by improving the functionality, identifying issues, and come up with or proposing solutions. Testing and output analysis are included to demonstrate the program's performance and accuracy.

Task 1: Horse Class Implementation

Summary:

The Horse class is created in order to represent each competitor in a simulated race. Each horse has a name, a unique symbol, a confidence level between 0 and 1, a distance counter, and a flag to indicate whether it has fallen during the race. The class provides methods for movement, resetting position, falling, and accessing or updating its state.

Encapsulation:

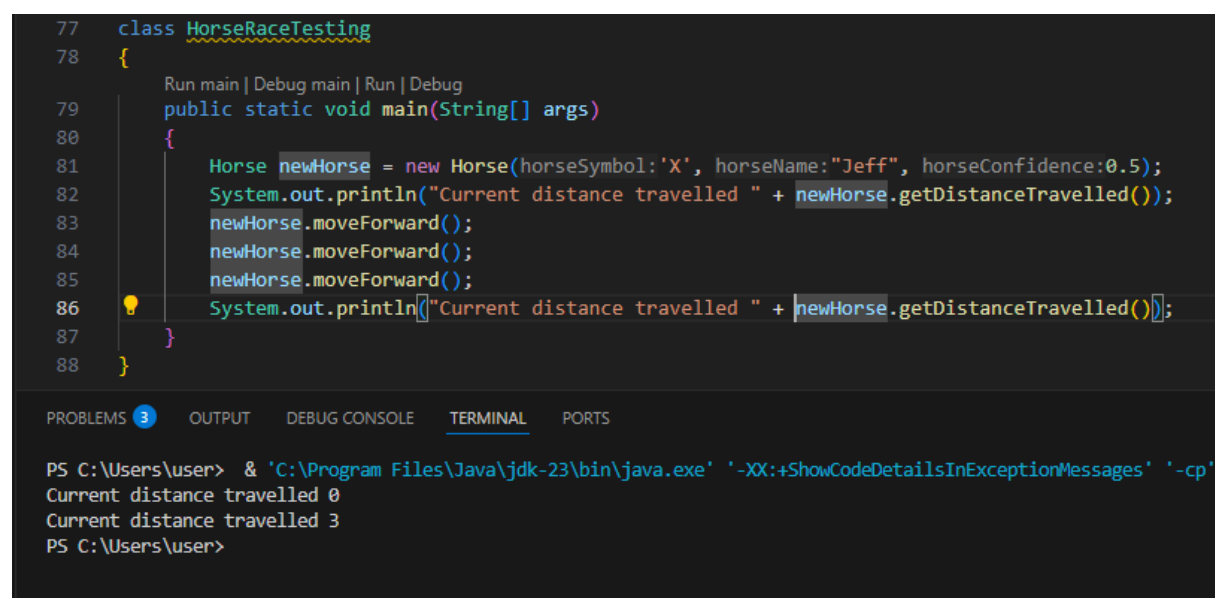
To ensure data protection and security, all fields in the Horse class are declared private. Public methods are used to interact with these fields safely. The use of accessor methods (getter methods) allows reading the values of the horse's name, symbol, confidence, distance travelled and fallen state, whereas mutator methods (setter methods) allow changes to specific data, such as updating the horse's symbol or setting a new confidence rating.

Other behaviour-related methods, such as `move Forward()`, `fall()`, and `goBackToStart()`, allow modification of the horse's state based on race events. Encapsulation ensures that no external code can modify the horse's data outside of the horse class, which therefore results in a more reliable, secure and maintainable program.

Testing and Results:

To test the Horse class, a Test class was written to simulate different actions a horse might perform during a race. The outputs were observed in the terminal.

The following screenshot shows a horse being created and then moved forward three times, confirming the distance is incremented correctly:



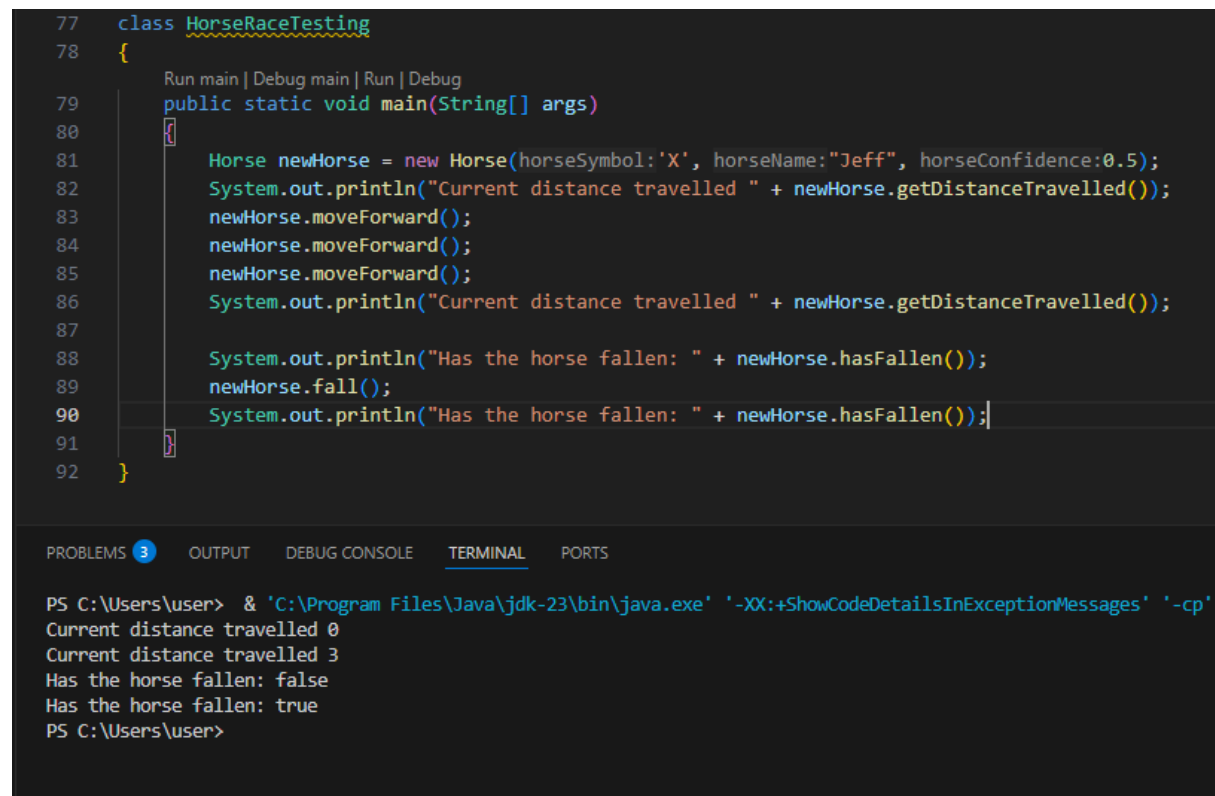
```
77 class HorseRaceTesting
78 {
    Run main | Debug main | Run | Debug
79 public static void main(String[] args)
80 {
81     Horse newHorse = new Horse(horseSymbol:'X', horseName:"Jeff", horseConfidence:0.5);
82     System.out.println("Current distance travelled " + newHorse.getDistanceTravelled());
83     newHorse.moveForward();
84     newHorse.moveForward();
85     newHorse.moveForward();
86     System.out.println("Current distance travelled " + newHorse.getDistanceTravelled());
87 }
88 }
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\user> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
Current distance travelled 0
Current distance travelled 3
PS C:\Users\user>
```

Figure 1: Testing `moveForward()` method. The distance increases by 1 with each call

This second test causes the horse to fall and confirms that the `hasFallen()` method correctly reflects this:



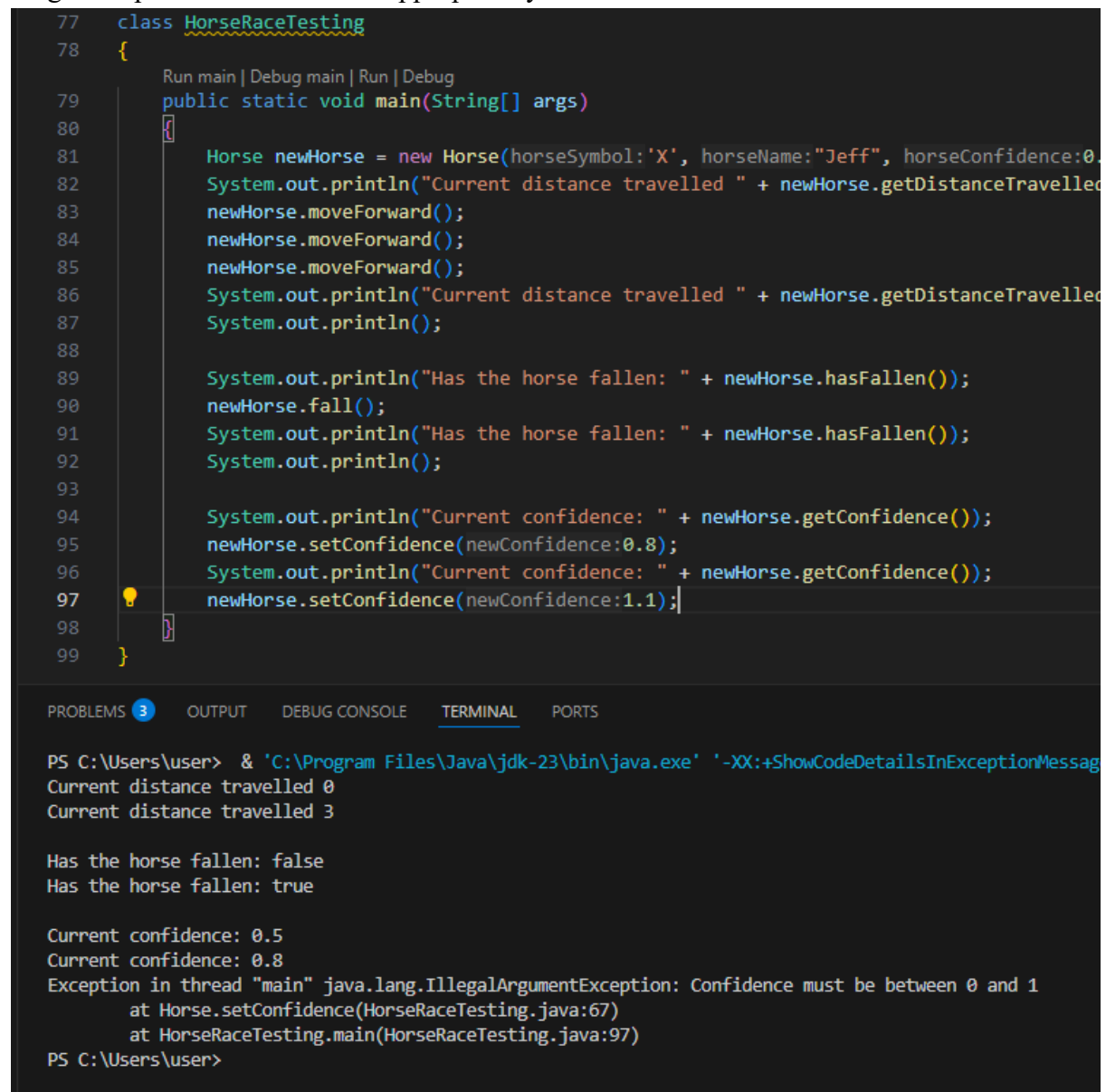
```
77 class HorseRaceTesting
78 {
79     Run main | Debug main | Run | Debug
80     public static void main(String[] args)
81     {
82         Horse newHorse = new Horse(horseSymbol:'X', horseName:"Jeff", horseConfidence:0.5);
83         System.out.println("Current distance travelled " + newHorse.getDistanceTravelled());
84         newHorse.moveForward();
85         newHorse.moveForward();
86         newHorse.moveForward();
87         System.out.println("Current distance travelled " + newHorse.getDistanceTravelled());
88         System.out.println("Has the horse fallen: " + newHorse.hasFallen());
89         newHorse.fall();
90         System.out.println("Has the horse fallen: " + newHorse.hasFallen());
91     }
92 }
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\user> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
Current distance travelled 0
Current distance travelled 3
Has the horse fallen: false
Has the horse fallen: true
PS C:\Users\user>
```

Figure 2: Testing the `fall()` method. The horse's `hasFallen` variable changes to `true` when `fall()` is called

The third screenshot shows that an attempt to set a confidence value outside the allowed range was prevented or handled appropriately:



```
77 class HorseRaceTesting
78 {
79     Run main | Debug main | Run | Debug
80     public static void main(String[] args)
81     {
82         Horse newHorse = new Horse(horseSymbol:'X', horseName:"Jeff", horseConfidence:0.5);
83         System.out.println("Current distance travelled " + newHorse.getDistanceTravelled());
84         newHorse.moveForward();
85         newHorse.moveForward();
86         newHorse.moveForward();
87         System.out.println("Current distance travelled " + newHorse.getDistanceTravelled());
88         System.out.println();
89         System.out.println("Has the horse fallen: " + newHorse.hasFallen());
90         newHorse.fall();
91         System.out.println("Has the horse fallen: " + newHorse.hasFallen());
92         System.out.println();
93         System.out.println("Current confidence: " + newHorse.getConfidence());
94         newHorse.setConfidence(newConfidence:0.8);
95         System.out.println("Current confidence: " + newHorse.getConfidence());
96         newHorse.setConfidence(newConfidence:1.1);
97     }
98 }
99 }
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\user> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessage'
Current distance travelled 0
Current distance travelled 3

Has the horse fallen: false
Has the horse fallen: true

Current confidence: 0.5
Current confidence: 0.8
Exception in thread "main" java.lang.IllegalArgumentException: Confidence must be between 0 and 1
    at Horse.setConfidence(HorseRaceTesting.java:67)
    at HorseRaceTesting.main(HorseRaceTesting.java:97)
PS C:\Users\user>
```

Figure 3: Testing the error handling. An attempt to set a confidence above 1 was made

Task 2: Race Class Improvements

Review of the Original Race Class:

The original Race class simulates a race between three horses. It uses a startRace() method to control when the race starts, it also assigns horses to lanes. The class also factors in confidence ratings for movement and the possibility of a horse falling.

(Original code can be seen in the appendix)

Issues Identified in the Original Race Class:

After reviewing and testing the provided code, the following problems and improvement areas were identified:

1. **No Winner Announcement:**

The startRace() method doesn't announce the winner when the race ends, which was a required feature.

2. **No Input Validation:**

There is no validation if horses haven't been added to all lanes. Running the race without all horses may cause errors (e.g., NullPointerException).

3. **Output Formatting Could Be Improved:**

The race display is functional but very minimal. Adding horse names and confidence values beside each track improves and matches the sample output provided.

4. **All three horses fall:**

There is not method that checks if all three horses have fallen. Running the race without this check could result in an infinite loop since none of the horses will finish the race.

Improvements made:

A winner announcement functionality was added to convey which horse won the race. Before each race starts, a final check is made to make sure each lane has a horse, if this is not the case an error message will pop up letting the user know. Each time the horses move forward by 1, the program checks if all horses have fallen or not, this way if all horses have fallen, there is a way for the race to stop and let the user know that all the horses have fallen. The display of the race and information has been improved by adding horse names and confidence values beside each track to match the sample output provided. The raceWon() method had it so that a horse would win the race if its distance travelled was equal to the total distance, however, this introduced an error where if a horse travelled more than the total distance, they wouldn't win. This was changed so that the condition for winning is `if(theHorse.getDistanceTravelled >= raceLength);`

Updated code:

```
private boolean raceWonBy(Horse theHorse)
{
    if (theHorse.getDistanceTravelled() >= raceLength)
    {
        System.err.println("And the winner is... " + theHorse.getName());
        return true;
    }
    else
    {
        return false;
    }
}
```

Figure 4: The winning horse's name is printed

```

    if (lane1Horse.hasFallen() && lane2Horse.hasFallen() && lane3Horse.hasFallen())
    {
        System.out.println(x:"All horses have fallen! The race is canceled.");
        finished = true;
    }

```

Figure 5: Check in case all horses have fallen

```

private void printLane(Horse theHorse)
{
    //calculate how many spaces are needed before
    //and after the horse
    int spacesBefore = theHorse.getDistanceTravelled();
    int spacesAfter = raceLength - theHorse.getDistanceTravelled();

    //print a | for the beginning of the lane
    System.out.print(c:'|');

    //print the spaces before the horse
    multiplePrint(aChar:' ',spacesBefore);

    //if the horse has fallen then print dead
    //else print the horse's symbol
    if(theHorse.hasFallen())
    {
        System.out.print(c:'\u2322');
    }
    else
    {
        System.out.print(theHorse.getSymbol());
    }

    //print the spaces after the horse
    multiplePrint(aChar:' ',spacesAfter);

    //print the | for the end of the track
    System.out.print("| " + theHorse.getName() + " (Current confidence " + theHorse.getConfidence() + ")");
}

```

Figure 6: Improved the display of the information as required

Testing and Outputs:

The Unicode provided by McRaceFace for clearing the terminal (System.out.print("\u000C");) and printing a red cross (System.out.print("\u2322");) was not supported and presented errors when running the race:

```

237     private void printRace()
238     {
239         System.out.print(c: '\u000C'); //clear the terminal window
240
241         multiplePrint(aChar: '=', raceLength+3); //top edge of track
242         System.out.println();
243
244     }

```

PROBLEMS 6 OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

|      # | Bob (Current confidence 0.2)
|      ? | Jeff (Current confidence 0.5)
|      ? | Chad (Current confidence 0.8)
=====
Q=====
|      # | Bob (Current confidence 0.2)
|      ? | Jeff (Current confidence 0.5)
|      ? | Chad (Current confidence 0.8)
=====
Q=====
|      # | Bob (Current confidence 0.2)
|      ? | Jeff (Current confidence 0.5)
|      ? | Chad (Current confidence 0.8)
=====
And the winner is... Bob
PS C:\Users\user>

```

Figure 7: Terminal didn't clear, and red cross didn't print correctly

The issue was not able to be fixed completely, instead ANSI was used to clear the terminal and print a red cross. This works across platforms like windows terminal and vs code, but not other platforms such as command prompt.

The rest of the code worked fine, and the race output was observed in the terminal and screenshots were taken of the final display, including the winning horse.

```

313 class HorseRaceSimulation
314 {
315     Run main | Debug main | Run | Debug
316     public static void main(String[] args)
317     {
318         Race race = new Race(distance:10);
319         race.addHorse(new Horse(horseSymbol:'#', horseName:"Bob", horseConfidence:0.2), laneN...1);
320         race.addHorse(new Horse(horseSymbol:'I', horseName:"Jeff", horseConfidence:0.5), laneN...2);
321         race.addHorse(new Horse(horseSymbol:'O', horseName:"Chad", horseConfidence:0.8), laneN...3);
322         race.startRace(); // This ensures `main` exists
323     }
324
325
326
327

```

PROBLEMS 12 OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

=====
|  X   | Bob (Current confidence 0.2)
|  I   | Jeff (Current confidence 0.5)
|  X   | Chad (Current confidence 0.8)
=====
And the winner is... Jeff
PS C:\Users\user>

```

```
313  class HorseRaceSimulation
314  {
315      Run main | Debug main | Run | Debug
316      public static void main(String[] args)
317      {
318          Race race = new Race(distance:10);
319          race.addHorse(new Horse(horseSymbol:'#', horseName:"Bob", horseConfidence:0.2), laneN...1);
320          race.addHorse(new Horse(horseSymbol:'I', horseName:"Jeff", horseConfidence:0.5), laneN...2);
321          race.addHorse(new Horse(horseSymbol:'O', horseName:"Chad", horseConfidence:0.8), laneN...3);
322          race.startRace(); // This ensures `main` exists
323      }
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

PROBLEMS 12 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
=====
| # | Bob (Current confidence 0.2)
| I | Jeff (Current confidence 0.5)
| O | Chad (Current confidence 0.8)
=====
And the winner is... Chad
PS C:\Users\user>
```

Conclusion

The development and refinement of the horse racing simulation was successful in demonstrating the use of object-oriented concepts such as encapsulation, privatisation, and modular design. The Horse class provides a clear, flexible structure for representing competitors, while improvements to the Race class improves the overall experience and reliability of the program. Further refinements could include detecting if there is a tie between horses, dynamic confidence updates, or a graphical display.

Appendix

The original McRaceFace code:

C:\> Users > user > AppData > Local > Temp > 63e5734c-3408-4306-92a3-1ea5582fc500_HorseRace Starter (1).zip.

```
1  import java.util.concurrent.TimeUnit;
2  import java.lang.Math;
3
4  /**
5   * A three-horse race, each horse running in its own lane
6   * for a given distance
7   *
8   * @author McRaceface
9   * @version 1.0
10  */
11  public class Race
12  {
13      private int raceLength;
14      private Horse lane1Horse;
15      private Horse lane2Horse;
16      private Horse lane3Horse;
17
18      /**
19       * Constructor for objects of class Race
20       * Initially there are no horses in the lanes
21       *
22       * @param distance the length of the racetrack (in metres/yards...)
23       */
24      public Race(int distance)
25      {
26          // initialise instance variables
27          raceLength = distance;
28          lane1Horse = null;
29          lane2Horse = null;
30          lane3Horse = null;
31      }
32
33      /**
34       * Adds a horse to the race in a given lane
35       *
36       * @param theHorse the horse to be added to the race
37       * @param laneNumber the lane that the horse will be added to
38       */
39      public void addHorse(Horse theHorse, int laneNumber)
40      {
41          if (laneNumber == 1)
42          {
43              lane1Horse = theHorse;
44          }
45          else if (laneNumber == 2)
46          {
47              lane2Horse = theHorse;
48          }
49          else if (laneNumber == 3)
50          {
51              lane3Horse = theHorse;
52          }
53          else
54          {
55              System.out.println("Cannot add horse to lane " + laneNumber + " because there is no such lane");
56          }
57      }
58
59      /**
60       * Start the race
61       * The horse are brought to the start and
62       * then repeatedly moved forward until the
63       * race is finished
64       */
65      public void startRace()
66      {
67          //declare a local variable to tell us when the race is finished
68          boolean finished = false;
69
70          //reset all the lanes (all horses not fallen and back to 0).
71          lane1Horse.goBackToStart();
72          lane2Horse.goBackToStart();
73          lane3Horse.goBackToStart();
74
75          while (!finished)
76          {
77              //move each horse
78              moveHorse(lane1Horse);
79              moveHorse(lane2Horse);
80              moveHorse(lane3Horse);
81
82              //print the race positions
83              printRace();
84          }
85      }
86  }
```

```

85         //if any of the three horses has won the race is finished
86         if ( raceWonBy(lane1Horse) || raceWonBy(lane2Horse) || raceWonBy(lane3Horse) )
87         {
88             finished = true;
89         }
90
91         //wait for 100 milliseconds
92         try{
93             TimeUnit.MILLISECONDS.sleep(timeout:100);
94         }catch(Exception e){}
95     }
96 }
97
98 /**
99  * Randomly make a horse move forward or fall depending
100  * on its confidence rating
101  * A fallen horse cannot move
102  *
103  * @param theHorse the horse to be moved
104  */
105 private void moveHorse(Horse theHorse)
106 {
107     //if the horse has fallen it cannot move,
108     //so only run if it has not fallen
109     if (!theHorse.hasFallen())
110     {
111         //the probability that the horse will move forward depends on the confidence;
112         if (Math.random() < theHorse.getConfidence())
113         {
114             theHorse.moveForward();
115         }
116
117         //the probability that the horse will fall is very small (max is 0.1)
118         //but will also will depends exponentially on confidence
119         //so if you double the confidence, the probability that it will fall is *2
120         if (Math.random() < (0.1*theHorse.getConfidence()*theHorse.getConfidence()))
121         {
122             theHorse.fall();
123         }
124     }
125 }
126
127 /**
128  * Determines if a horse has won the race
129  *
130  * @param theHorse The horse we are testing
131  * @return true if the horse has won, false otherwise.
132  */
133 private boolean raceWonBy(Horse theHorse)
134 {
135     if (theHorse.getDistanceTravelled() == raceLength)
136     {
137         return true;
138     }
139     else
140     {
141         return false;
142     }
143 }
144
145 /**
146  * Print the race on the terminal
147  */
148 private void printRace()
149 {
150     System.out.print(c: '\u000C'); //clear the terminal window
151
152     multiplePrint(aChar: '=',raceLength+3); //top edge of track
153     System.out.println();
154
155     printLane(lane1Horse);
156     System.out.println();
157
158     printLane(lane2Horse);
159     System.out.println();
160
161     printLane(lane3Horse);
162     System.out.println();
163
164     multiplePrint(aChar: '=',raceLength+3); //bottom edge of track
165     System.out.println();
166 }
167
168 /**

```

```

169     * print a horse's lane during the race
170     * for example
171     * |           X           |
172     * to show how far the horse has run
173     */
174 private void printLane(Horse theHorse)
175 {
176     //calculate how many spaces are needed before
177     //and after the horse
178     int spacesBefore = theHorse.getDistanceTravelled();
179     int spacesAfter = raceLength - theHorse.getDistanceTravelled();
180
181     //print a | for the beginning of the lane
182     System.out.print(c: '|');
183
184     //print the spaces before the horse
185     multiplePrint(aChar: ' ',spacesBefore);
186
187     //if the horse has fallen then print dead
188     //else print the horse's symbol
189     if(theHorse.hasFallen())
190     {
191         System.out.print(c: '\u2322');
192     }
193     else
194     {
195         System.out.print(theHorse.getSymbol());
196     }
197
198     //print the spaces after the horse
199     multiplePrint(aChar: ' ',spacesAfter);
200
201     //print the | for the end of the track
202     System.out.print(c: '|');
203 }
204
205
206 /**
207  * print a character a given number of times.
208  * e.g. printmany('x',5) will print: xxxxx
209  *
210  * @param aChar the character to Print
211  */
212 private void multiplePrint(char aChar, int times)
213 {
214     int i = 0;
215     while (i < times)
216     {
217         System.out.print(aChar);
218         i = i + 1;
219     }
220 }
221
222

```